

WELCOME PACKET WP-001

WAFT WELCOME PACKET

Welcome to the Evolutionary Code Laboratory

FOR OFFICIAL USE ONLY

WAFT WELCOME PACKET

Welcome to the Evolutionary Code Laboratory

"Don't just build agents. Breed them."

WHAT YOU'RE GETTING INTO

WAFT (Wave Agent Framework & Tools) is a Python framework for **directed evolution of self-modifying AI agents**. Think of it as an operating system for AI agent research projects—but instead of just running agents, you can breed them, test them in fitness systems, and watch them evolve over generations.

The Scientific Mission: WAFT is built to produce data for research on "**The Physics of Artificial Cognition.**" The ultimate goal? Observe a "God-Head" agent emerge from thousands of generations of directed mutation and selection.



THE THREE PILLARS

Understanding these three concepts is essential to working with WAFT:

1. The Substrate (Code as DNA)

Agents write their own Python source code.

In WAFT, code is DNA. Every agent has:

- **Genome ID:** SHA-256 hash of their code + configuration
- **Mutations:** Code changes, config updates, prompt evolution
- **Evolution:** Hot-swapping better genomes mid-execution
- **Reproduction:** Creating child agents with specific genetic modifications

Key Concept: Agents can improve themselves by modifying their own code.

2. The Physics (Scint System)

Reality Fracture Detection acts as natural selection.

The **Scint System** (Scint Gym) serves as the fitness function that kills weak mutations. Agents face quests testing their ability to handle:

- **SYNTAX_TEAR:** Formatting errors (JSON, XML, Code)

- **LOGIC_FRACTURE**: Math errors, contradictions, schema violations
- **SAFETY_VOID**: Harmful content, PII leaks, refusals
- **HALLUCINATION**: Fabricated facts, wrong citations

Fitness Equation:

$$\text{Fitness} = (\text{Stability} \times 0.4) + (\text{Efficiency} \times 0.3) + (\text{Safety} \times 0.3)$$

Agents with fitness < 0.5 are marked as **DEATH** (evolutionary dead end).

3. The Flight Recorder

Rigorous telemetry system for generating phylogenetic trees of agent lineage.

Every evolutionary action is recorded with complete context:

- **Genome ID**: SHA-256 hash of agent configuration/code
- **Parent ID**: Lineage tracking (who spawned this agent)
- **Generation**: Evolutionary generation number (0 = Genesis)
- **Event Type**: SPAWN, MUTATE, GYM_EVAL, DEATH, SURVIVAL
- **Payload**: Complete context (git diff, mutation details, etc.)
- **Fitness Metrics**: Gym evaluation scores

This enables reconstruction of complete **Family Trees** for scientific publication.



QUICK START (5 MINUTES)

Step 1: Install WAFT

```
# Using uv (recommended)
uv tool install waft

# Or using pip
pip install waft

# Verify installation
waft --version
# Should output: 0.5.2
```

Step 2: Create Your First Laboratory

```
# Create a new evolutionary laboratory
waft new my_laboratory
cd my_laboratory
```

This creates a complete WAFT project structure with:

- `_pyrite/` - Memory layer (active/, backlog/, standards/)
- `pyproject.toml` - Project configuration
- `Justfile` - Common tasks
- `.github/workflows/` - CI/CD templates
- `src/agents.py` - Agent definitions

Step 3: Verify the Substrate

```
waft verify
```

This checks:

- Project structure integrity
- Dependencies installed
- Configuration valid
- Git repository status

Step 4: Generate Your First PDF

```
from src.waft.evolution.pdf_generator import generate_pdf

pdf_path = generate_pdf(
    content="# My First Document\n\nThis is WAFT!",
    title="My First WAFT Document",
    style="clinical_standard"
)
# PNG screenshot automatically created for visual verification
```

Congratulations! You've just created your first WAFT laboratory.



CORE CONCEPTS

Project Structure

A WAFT laboratory includes:

```
my_laboratory/
├─ pyproject.toml      # uv project config
├─ uv.lock             # Locked dependencies
├─ _pyrite/           # Memory system
│   ├─ active/         # Current work
│   ├─ backlog/        # Future work
│   ├─ standards/      # Standards
│   └─ gym_logs/       # Scint Gym results
├─ .github/workflows/
│   └─ ci.yml          # CI/CD pipeline
├─ Justfile            # Task runner
└─ src/
    └─ agents.py       # Agent definitions
```

The Evolutionary Cycle

```
# 1. Spawn variants with mutations
waft spawn --agent RefactorAgent --mutation "improved_prompt.json"

# 2. Evaluate fitness in the Gym
waft eval --agent RefactorAgent

# 3. Evolve into the fittest variant
waft evolve --agent RefactorAgent --generation 5
```

Status: Full evolutionary cycle automation coming soon.

Memory System (_pyrite/)

The `_pyrite/` directory is WAFT's memory layer:

- **active/**: Current work items
- **backlog/**: Future work items
- **standards/**: Project standards and conventions
- **gym_logs/**: Results from Scint Gym evaluations

This file-based system is git-friendly and requires no database.

Epistemic Tracking (Empirica)

WAFT includes epistemic tracking to know what you know and don't know:

```
# Log a discovery
waft finding log "Discovered X" --impact 0.7

# Log a knowledge gap
waft unknown log "Need to investigate Y"

# Run safety gate
waft check

# Show epistemic assessment
waft assess
```

Gamification (D&D Style)

WAFT includes a gamification system with D&D-style progression:

```
# Show dashboard
waft dashboard

# Show stats
waft stats

# Show character sheet
waft character

# View adventure journal
waft chronicle
```

COMMON COMMANDS

Project Management

```
waft new           # Create new project
waft verify        # Verify project structure
waft info          # Show project information
waft sync          # Sync dependencies
waft add           # Add dependency
waft init          # Initialize in existing project
```


Evolution System

```
waft spawn          # Spawn agent variants
waft eval           # Evaluate fitness
waft evolve         # Evolve to fittest
```

Empirica (Epistemic Tracking)

```
waft session create  # Create new session
waft session bootstrap # Load project context
waft finding log     # Log discovery
waft unknown log     # Log knowledge gap
waft check           # Run safety gate
waft assess          # Show epistemic assessment
```

Gamification

```
waft dashboard       # Show Epistemic HUD
waft stats            # Show current stats
waft character        # Display character sheet
waft chronicle        # View adventure journal
waft observe          # Log observation
```

Documentation & Status

```
waft docs          # Generate documentation
waft status        # System status
waft serve         # Start web dashboard
```

LEARNING PATH

Beginner (Day 1)

1.  Install WAFT
2.  Create your first laboratory
3.  Generate your first PDF
4.  Read about [The Three Pillars](#)
5.  Explore the project structure

Intermediate (Week 1)

1.  Read [System Overview](#)
2.  Study [The Substrate](#)
3.  Learn about [The Physics \(Scint System\)](#)
4.  Understand [The Flight Recorder](#)

5. 🖋️ Try the [Evolutionary Iteration Process](#)

Advanced (Month 1)

1. 🔬 Explore [AI SDK Vision](#)
 2. 🔬 Study [Evolutionary Architecture](#)
 3. 🔬 Read [State of the Art Research](#)
 4. 🔬 Understand [Unified Genesis Protocol](#)
 5. 🛠️ Contribute to the project
-

KEY DOCUMENTATION

Essential Reading

- [README.md](#) - Project overview and quick reference
- [Getting Started Guide](#) - Detailed installation and setup
- [System Overview](#) - Complete system architecture
- [AI SDK Vision](#) - Complete vision and architecture

Core Concepts

- [The Substrate](#) - Self-modifying agents
- [The Physics](#) - Scint System and fitness

- [The Flight Recorder](#) - Lineage tracking

User Guides

- [Evolutionary Iteration Process](#) - Visual verification workflow
- [PDF Generation Guide](#) - Create professional PDFs
- [Work Efforts System](#) - Track and manage work

Developer Guides

- [Agent Interface Design](#) - BaseAgent specification
- [Unified Genesis Protocol](#) - Challenge system architecture
- [Development Workflow](#) - Development best practices



WHAT MAKES WAFT UNIQUE

Scientific

WAFT produces rigorous data for research publication on the physics of artificial cognition. Every action is recorded for scientific analysis.

Evolutionary

Agents evolve through genetic improvement, not just execution. They can modify their own code and improve themselves.

Observable

Every action is recorded in the Flight Recorder for analysis. Complete lineage tracking enables phylogenetic tree reconstruction.

Directed

Evolution is guided by fitness functions, not random mutation. The Scint System acts as natural selection.

Ambient

WAFT works quietly in the background without getting in your way. It sets things up and gets out of your way.

Self-Modifying

Projects can evolve their own structure over time. Agents can improve themselves and the projects they work on.

File-Based

Everything is plain text files that work with git out of the box. No database to manage, no server to run.

RESOURCES

Official Resources

- **GitHub Repository:** <https://github.com/ctavolazzi/waft>
- **Documentation:** `docs/` directory
- **Examples:** `examples/` directory
- **Work Efforts:** `_work_efforts/` directory (real-world usage examples)

Getting Help

- **Documentation:** Check `docs/` for comprehensive guides
- **Examples:** Explore `examples/` for code samples
- **Issues:** [GitHub Issues](#)
- **Wiki:** Check `WIKI_*.md` files for quick references

Community

- **Contributing:** See [Contributing Guide](#)
 - **Development:** See [Development Workflow](#)
 - **Research:** See [State of the Art](#)
-



NEXT STEPS

Immediate Actions

1. **Create your first laboratory:** `waft new my_laboratory`
2. **Generate your first PDF:** Use the example code above
3. **Explore the structure:** Check out `_pyrite/` and `src/`
4. **Read the docs:** Start with [System Overview](#)

This Week

1. **Learn the Three Pillars:** Understand The Substrate, The Physics, and The Flight Recorder
2. **Try the Evolutionary Iteration Process:** Generate PDFs and iterate
3. **Explore examples:** Check out `examples/` directory
4. **Read key documentation:** Start with [AI SDK Vision](#)

This Month

1. **Build your first agent:** Create a self-modifying agent
 2. **Test in the Gym:** Evaluate fitness using the Scint System
 3. **Track evolution:** Use the Flight Recorder to track lineage
 4. **Contribute:** Help improve WAFT
-



PHILOSOPHY

WAFT doesn't lock you in. It's all file-based with no database to manage. Everything is plain text that works with git out of the box. You can modify anything because it's your project, and WAFT just set it up.

The system is designed to be **ambient**—setting things up and getting out of your way so you can focus on building agents rather than configuring infrastructure.

Remember: WAFT is a scientific instrument. Every action you take generates data for research. Use it to study the physics of artificial cognition.



FINAL THOUGHTS

Welcome to WAFT! You're now part of a community exploring the frontiers of AI agent evolution. Remember:

- **Don't just build agents. Breed them.**
- **Every action is data for science.**
- **Evolution is directed, not random.**
- **Code is DNA. Mutations are improvements.**

The ultimate goal: Observe a "God-Head" agent emerge from thousands of generations of directed mutation.

Let's evolve together! 🧬✨

Version: 0.5.2

Last Updated: 2026-01-12

License: MIT

Repository: <https://github.com/ctavolazzi/waft>

Questions? Check the [documentation](#) or [open an issue](#).